

Lab 5: Building a Simple Workflow Engine – Delegates, Events, and Lambdas

Goal

Create a lightweight, extensible workflow system using delegates, lambda expressions, and events. The workflow engine will execute a sequence of steps, emit lifecycle events, and support async operations and logging.

Step-by-Step Instructions

✅ Step 1: Create the Project

1. Open Visual Studio or your preferred C# development environment.
2. Create a new Console App project named `WorkflowEngineLab`.

✅ Step 2: Define the Workflow Delegate

```
delegate Task WorkflowStepAsync();
```

This delegate represents an asynchronous step in the workflow.

✅ Step 3: Define the Workflow Engine Class

Create the core class that stores steps and exposes events:

```
public class WorkflowEngine
{
    public event EventHandler<string> StepStarted;
    public event EventHandler<string> StepCompleted;
    public event EventHandler<string> StepFailed;

    private List<(string Name, WorkflowStepAsync Step)> steps = new();

    public void AddStep(string name, WorkflowStepAsync step)
    {
        steps.Add((name, step));
    }

    public async Task ExecuteAsync()
```

```

{
    foreach (var (name, step) in steps)
    {
        StepStarted?.Invoke(this, name);
        try
        {
            await step();
            StepCompleted?.Invoke(this, name);
        }
        catch
        {
            StepFailed?.Invoke(this, name);
        }
    }
}
}

```

✓ Step 4: Add Sample Workflow Steps

Use lambda expressions to add simple steps:

```

var engine = new WorkflowEngine();

engine.AddStep("DownloadFile", async () => {
    await Task.Delay(500);
    Console.WriteLine("Downloading file...");
});

engine.AddStep("ProcessData", async () => {
    await Task.Delay(500);
    Console.WriteLine("Processing data...");
});

engine.AddStep("UploadResults", async () => {
    await Task.Delay(500);
    Console.WriteLine("Uploading results...");
});

```

✓ Step 5: Subscribe to Lifecycle Events

Log each event using lambda expressions:

```
engine.StepStarted += (s, name) => Console.WriteLine($"[START] {name}");
engine.StepCompleted += (s, name) => Console.WriteLine($"[DONE] {name}");
engine.StepFailed += (s, name) => Console.WriteLine($"[FAIL] {name}");
```

✅ Step 6: Run the Workflow

In `Main()`, call:

```
await engine.ExecuteAsync();
```

Make `Main` asynchronous:

```
static async Task Main(string[] args)
```

💡 Stretch Goals (Included in Base Lab)

🔄 Conditional Execution (Optional Steps)

Modify `AddStep` to accept a predicate:

```
public void AddConditionalStep(string name, WorkflowStepAsync step, Func<bool> condition)
{
    if (condition())
    {
        steps.Add((name, step));
    }
}
```

Usage:

```
bool debug = true;
engine.AddConditionalStep("DebugLog", async () => {
    Console.WriteLine("[DEBUG] Running debug step");
}, () => debug);
```

🔄 Rollback or Undo Steps

Add an optional rollback delegate:

```
public class WorkflowStepWithRollback
{
    public string Name;
    public WorkflowStepAsync Step;
    public WorkflowStepAsync Rollback;
}
```

Update the engine to handle rollback logic (left as design challenge for students).

Logging with a Custom Logger

Inject a logger using a delegate:

```
delegate void LogMessage(string message);
```

Add a logger property:

```
public LogMessage Logger { get; set; } = Console.WriteLine;
```

Use it in event triggers:

```
Logger?.Invoke($"[INFO] Step '{name}' started");
```

What You Learned

- How to define and use asynchronous delegates
- How to model a sequence of steps as pluggable components
- How to wire up event-driven logic
- How to combine lambdas, delegates, and events for workflow design
- How to add extensibility with optional features like conditions, logging, and rollbacks

Wrap-Up Challenge

Add an interactive mode that allows a user to choose which steps to execute at runtime using the console.